

# CD Lab Record 1

## Conversion of DFA to Regular Expression

### Aim

To convert a given Deterministic Finite Automaton (DFA) into an equivalent Regular Expression using a formal mathematical method based on state-wise recurrence relations.

### Objective

The objective of this experiment is to understand the theoretical equivalence between Deterministic Finite Automata (DFA) and Regular Expressions (RE), and to systematically derive a regular expression that represents the same language as the given DFA using a rigorous algorithmic approach.

### Theory

Regular Expressions and Finite Automata are two equivalent ways of describing regular languages. According to Kleene's Theorem, a language is regular if and only if it can be accepted by a finite automaton and described by a regular expression.

In this experiment, the conversion from DFA to Regular Expression is performed using the McNaughton–Yamada construction, which is a formal and systematic method. Instead of eliminating states informally, this method defines a recurrence relation over all pairs of states.

Let  $R_{ij}^k$  denote the regular expression representing all strings that take the automaton from state  $q_i$  to state  $q_j$ , using only the states  $\{q_1, q_2, \dots, q_k\}$  as intermediate states.

### Base Case ( $k = 0$ )

When no intermediate states are allowed:

- If  $i \neq j$ ,

$$R_{ij}^0 = \{a \mid \delta(q_i, a) = q_j\}$$

- If  $i = j$ ,

$$R_{ii}^0 = \{a \mid \delta(q_i, a) = q_i\} \cup \{\epsilon\}$$

This represents only direct transitions, along with the empty string when the start and end states are the same.

## Recursive Case ( $k > 0$ )

When state  $q_k$  is allowed as an intermediate state:

$$R_{ij}^k = R_{ij}^{k-1} \cup R_{ik}^{k-1} (R_{kk}^{k-1})^* R_{kj}^{k-1}$$

This equation considers two possibilities:

1. Paths that do not pass through  $q_k$
2. Paths that pass through  $q_k$  one or more times

The Kleene star represents repeated looping at state  $q_k$ .

## Final Regular Expression

If  $q_s$  is the start state and  $F$  is the set of final states, then the required regular expression is:

$$\bigcup_{q_f \in F} R_{sf}^n$$

where  $n$  is the total number of states in the DFA.

## Algorithm (High-Level)

1. Read the DFA details: number of states, symbols, start state, transitions, and final states.
2. Initialize the base expressions  $R_{ij}^0$  using direct transitions and  $\epsilon$ .
3. Iteratively compute  $R_{ij}^k$  for  $k = 1$  to  $n$  using the recurrence relation.
4. Combine expressions from the start state to all final states.
5. Output the resulting regular expression.

# Observations

- The generated regular expression may not always be in its simplest form.
- Different regular expressions can represent the same regular language.
- Algebraic simplification of regular expressions is not part of the McNaughton–Yamada construction and is optional.
- Correct handling of  $\epsilon$  (empty string) and  $\emptyset$  (empty language) is crucial for correctness.

## Result

The given DFA was successfully converted into an equivalent regular expression using the McNaughton–Yamada recurrence method. The output regular expressions correctly represent the same languages accepted by the original DFAs, thereby validating the theoretical equivalence between DFAs and Regular Expressions.

## Conclusion

This experiment reinforced the theoretical foundation of regular languages and demonstrated a rigorous algorithmic approach for converting a DFA into a regular expression. The McNaughton–Yamada method provides a systematic and provably correct way to perform this conversion, highlighting the deep connection between automata theory and formal language representation.